LAPORAN PRAKTIKUM PEKAN 3

STRUKTUR DATA

“STACK”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan Stack menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2026 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 18 April 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

BAB II PEMBAHASAN 2

2.1 Mengenal Stack pada Java 2

2.2 Karakteristik dan Operasi Stack 3

2.3 Implementasi Stack 4

2.3.1 Class NilaiMaksimum_2511533011 4

2.3.2 Class SiswaStack_2511533011 6

2.3.3 Class stackArray_2511533011 9

2.3.4 Class stackArrayDriver_2511533011 11

2.3.5 Class StackPostfix_2511533011 13

BAB III PENUTUP 15

3.1 Kesimpulan 15

DAFTAR PUSTAKA 16

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, efisiensi pengelolaan data merupakan aspek krusial dalam pembangunan perangkat lunak. Salah satu struktur data linear yang fundamental adalah **Stack** (tumpukan). Stack bekerja dengan prinsip **LIFO (*Last In First Out*)**, di mana data yang terakhir dimasukkan ke dalam struktur akan menjadi data yang pertama kali dikeluarkan.

Konsep stack sangat menyerupai tumpukan benda di dunia nyata, seperti tumpukan piring atau buku. Dalam implementasi komputasi, stack digunakan dalam berbagai fungsi penting seperti proses *undo/redo* pada editor teks, pengelolaan memori (*call stack*) pada bahasa pemrograman, hingga algoritma pencarian seperti *Depth First Search* (DFS). Memahami cara kerja stack, baik melalui implementasi *Array* maupun *Linked List*, sangat penting bagi mahasiswa Informatika untuk memperkuat logika pemrograman dan struktur data.

1.2 Tujuan Praktikum

1. Mahasiswa mampu memahami karakteristik dan prinsip kerja struktur data Stack (LIFO).
2. Mahasiswa dapat mengimplementasikan operasi-operasi dasar pada Stack seperti **push, pop, peek, isEmpty,** dan **isFull.**
3. Mahasiswa mampu membandingkan efisiensi implementasi Stack menggunakan media penyimpanan statis (Array) dan dinamis (Linked List).
4. Mahasiswa mampu menerapkan struktur data Stack untuk menyelesaikan permasalahan logika tertentu (missal : pembalikan string atau konversi bilangan).

BAB II
PEMBAHASAN

2.1 Mengenal Stack pada Java

Secara sederhana, stack adalah **struktur data linier** yang digunakan untuk menyimpan sekumpulan objek atau variabel. Setiap penambahan data (push) dan penghapusan data (pop) terjadi di *ujung atas* tumpukan. Sifat khas stack adalah **LIFO (Last In, First Out)**: data terakhir yang masuk akan menjadi data pertama yang keluar.

Dalam dunia nyata, analogi stack sering dijumpai pada tumpukan buku atau piring. Misalnya, jika kamu menumpuk buku A, B, C, buku C (yang terakhir diletakkan) harus dikeluarkan dulu untuk mengambil buku yang lebih bawah. Begitu pula dalam pemrograman, elemen yang paling baru ditambahkan harus dikeluarkan terlebih dahulu.

Analoginya seperti sebuah tumpukan piring kotor di dapur, piring kotor terakhir yang di letakkan akan berada pada bagian paling atas. Ketika ingin mencucinya, biasanya piring paling atas yang akan dicuci karena mudah dijangkau. Inilah cara kerja Stack di pemrograman.

Stack sering digunakan dalam berbagai konteks pemrograman. Misalnya pada saat **panggilan fungsi** (call stack) dan **rekursi**, setiap pemanggilan fungsi disimpan dalam stack hingga fungsi tersebut selesai dieksekusi. Contoh lain adalah **undo/redo** di aplikasi, evaluasi ekspresi matematika, serta pengecekan kesetaraan tanda kurung. Semua kasus ini memanfaatkan sifat LIFO stack agar data terkelola dengan teratur.

2.2 Karakteristik dan Operasi Stack

Karakteristik utama stack adalah aturan LIFO. Berikut operasi dasar yang biasa diterapkan pada stack:

- **Push:** Menambahkan elemen baru ke *bagian atas* stack. Misalnya, menumpuk buku baru di atas tumpukan.
- **Pop:** Menghapus elemen di *puncak* stack dan mengembalikan nilainya. Artinya, buku yang paling atas diambil keluar.
- **Peek/Top:** Melihat (mengintip) elemen teratas tanpa menghapusnya. Seperti mengintip buku paling atas tanpa mengambilnya.
- **IsEmpty:** Memeriksa apakah stack kosong atau tidak. Cocok seperti mengecek apakah masih ada piring di tumpukan.
- **Size:** Mengembalikan jumlah elemen dalam stack. Berguna untuk mengetahui seberapa tinggi tumpukan saat ini.

Operasi-operasi di atas dijalankan dari satu sisi stack (ujung atas). Contohnya, operasi **push** tidak boleh menambah data di bagian bawah tumpukan, dan **pop** hanya bisa menghapus dari atas. Hal ini membuat stack berbeda dengan struktur lain (misalnya queue) yang dapat menghapus dari ujung depan. Misalnya pada *stack*, hanya elemen terakhir yang dapat diakses atau dikeluarkan tanpa membongkar seluruh data di bawahnya.

Karakteristik LIFO juga berarti stack tidak mendukung *akses acak*. Untuk mengambil elemen tertentu, kita harus mengeluarkan semua elemen di atasnya terlebih dahulu. Kekurangan ini merupakan batasan yang wajar karena desain stack memang fokus pada penyimpanan sementara untuk urutan tugas tertentu (misalnya dalam pemanggilan fungsi).

2.3 Implementasi Stack

2.3.1 Class NilaiMaksimum_2511533011

```java
package pekan3_2511533011;
import java.util.Stack;

public class NilaiMaksimum_2511533011 {
    public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack <Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue;
    }
    public static void main(String[] args) {
        Stack <Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack " + s);
        System.out.println("Stack teratas " + s.peek());
        System.out.println("Nilai maksimum " + max(s));
    }
}
```

Gambar 2.3.1

Class ini mengimplementasikan algoritma untuk mencari nilai terbesar dalam sebuah tumpukan tanpa menghilangkan data asli di dalamnya.

1. Penggunaan Library java.util.Stack

Pada praktikum ini, implementasi tidak dilakukan secara manual, melainkan memanfaatkan *collection* yang sudah disediakan oleh Java. Hal ini menunjukkan efisiensi dalam penggunaan fungsi-fungsi standar seperti push(), pop(), peek(), dan isEmpty().

2. Algoritma Pencarian Nilai Maksimum

Bagian paling krusial dalam class ini adalah metode static int max(Stack<Integer> s). Logika yang digunakan adalah:

- **Penggunaan Stack Cadangan (backup)**: Karena sifat Stack yang destruktif saat melakukan pop(), digunakan stack backup untuk menyimpan sementara elemen yang dikeluarkan agar data asli tidak hilang.
- **Iterasi dan Perbandingan**: Menggunakan perulangan while (!s.isEmpty()) untuk mengambil setiap elemen. Nilai dibandingkan menggunakan Math.max(maxValue, next) untuk menentukan angka tertinggi.
- **Restorasi Data**: Setelah nilai maksimum ditemukan, semua elemen di dalam backup dikembalikan lagi ke stack asal s agar urutan data tetap terjaga seperti semula.

3. Analisis Alur Eksekusi (Tracing)

Dalam metode main, diberikan input data berupa 70, 12, dan 20.

- Saat max(s) dipanggil, program akan membongkar tumpukan.
- Meskipun 70 berada di paling bawah (karena di-*push* pertama), algoritma tetap bisa menemukannya karena seluruh isi stack dipindahkan ke backup.
- Output yang dihasilkan menunjukkan nilai maksimum adalah **70**.

2.3.2 Class SiswaStack_2511533011

```java
package pekan3_2511533011;
import java.util.ArrayList;

class Siswa_2511533011 {
    String nama;
    int nim;

    public Siswa_2511533011(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim : " + nim + ", Nama : " + nama;
    }

}
public class SiswaStack_2511533011 {
    private ArrayList<Siswa_2511533011> stack;

    public SiswaStack_2511533011() {
        stack = new ArrayList<>();
    }

    public void push(Siswa_2511533011 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511533011 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }
    public Siswa_2511533011 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;

    }
    public boolean isEmpty() {
```

```java
        return stack.isEmpty();
    }
    public void tampilkanSiswa() {
        for (int i = stack.size() -1; i >= 0; i--) {
            System.out.println(stack.get(i));

        }
    }
    public static void main(String[] args) {
        SiswaStack_2511533011 studentStack = new SiswaStack_2511533011();

        Siswa_2511533011 mhs1 = new Siswa_2511533011("Ali", 1);
        Siswa_2511533011 mhs2 = new Siswa_2511533011("Boby", 2);
        Siswa_2511533011 mhs3 = new Siswa_2511533011("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack: ");
        studentStack.tampilkanSiswa();

        System.out.println("siswa teratas " + studentStack.peek());
        System.out.println("Mengeluarkan siswa teratas dari stack : " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop : ");
        studentStack.tampilkanSiswa();

    }
}
```

Gambar 2.3.2

Class ini berperan sebagai POJO ***(Plain Old Java Object)*** atau model data.

- **Atribut:** Memiliki dua atribut utama, yaitu nama (String) dan nim (int).
- **Method toString():** Dilakukan *override* pada method ini agar saat objek dicetak (misalnya pada System.out.println), yang muncul bukan alamat memori, melainkan format string yang rapi berisi NIM dan Nama mahasiswa.

1. Implementasi Operasi Stack

- **Metode push()**: Menggunakan stack.add(mhs), yang secara otomatis menambahkan elemen ke indeks terakhir (ujung tumpukan).
- **Metode pop()**: Mengambil data dari stack.size() - 1. Ini sangat penting karena dalam konsep Stack, elemen yang keluar adalah yang berada di posisi paling akhir (LIFO).
- **Metode peek()**: Hanya melihat data di indeks terakhir tanpa menghapusnya dari ArrayList.

2. Logika tampilkanSiswa()

Pada metode ini, for (int i = stack.size() - 1; i >= 0; i--). Ini adalah cara yang tepat untuk memvisualisasikan Stack. Data dicetak dari indeks terbesar ke terkecil agar pengguna bisa melihat siapa yang berada di posisi "paling atas" (elemen terakhir yang di-*push*).

3. Keunggulan Menggunakan ArrayList

- **Ukuran Dinamis:** Stack tidak akan penuh (*overflow*) selama memori komputer masih mencukupi, karena ArrayList akan otomatis melakukan *resize* ukurannya.
- **Kemudahan Manajemen:** Kita tidak perlu mengelola *pointer* secara manual seperti pada *Linked List* atau mengecek indeks secara kaku seperti pada *Array* statis.

2.3.3 Class stackArray_2511533011

```java
package pekan3_2511533011;

public class stackArray_2511533011 {
    static final int MAX = 1000;
    int top_3011;
    int a[] = new int[MAX];
    boolean isEmpty()
    {
            return (top_3011 < 0);

    }
    stackArray_2511533011()
    {
        top_3011 = -1;
    }
    boolean push_3011(int x)
    {
        if (top_3011 >= (MAX -1) ) {
            System.out.println("StackOverflow");
            return false;
        }
        else {
            a[++top_3011] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }

    }
    int pop_3011()
    {
        if (top_3011 <= 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top_3011--];
            return x;
        }

    }
    int peek_3011()
```

```java
    {
        if (top_3011 < 0) {
            System.out.println("Stuck Underflow");
            return 0;

        }
        else {
            int x = a[top_3011];
            return x;
        }

    }
    void print() {
        for (int i = top_3011; i>-1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Class ini mengimplementasikan struktur data Stack dengan menggunakan array sebagai media penyimpanan data. Berbeda dengan penggunaan ArrayList, di sini ukuran memori ditentukan secara statis di awal.

1. Inisialisasi dan Variabel top_3011

Variabel top_3011 berfungsi sebagai *pointer* yang menunjukkan indeks elemen teratas saat ini.

- **Kondisi Awal:** Pada constructor, top_3011 diinisialisasi dengan nilai -1. Hal ini menandakan bahwa tumpukan sedang kosong (karena indeks array dimulai dari 0).
- **Konstanta MAX:** Nilai 1000 ditetapkan sebagai batas maksimum tumpukan, yang menjadi batasan utama dalam implementasi array statis.

2. Mekanisme Push (Penambahan Data)

Dalam metode push_3011(int x), dilakukan pengecekan kondisi sebelum data dimasukkan:

- **Pencegahan Stack Overflow:** Jika top_3011 sudah mencapai MAX - 1, maka tumpukan dianggap penuh dan sistem akan menolak data baru.
- **Pre-increment:** Penggunaan kode a[++top_3011] = x menunjukkan bahwa nilai top_3011 dinaikkan terlebih dahulu, baru kemudian data disimpan pada indeks tersebut.

3. Mekanisme Pop dan Peek (Pengambilan Data)

- **Metode pop_3011():** Selain mengambil nilai, metode ini melakukan *post-decrement* (top_3011--). Artinya, data diambil dulu, baru posisi pointer diturunkan.
- **Metode peek_3011():** Berbeda dengan pop, peek hanya mengembalikan nilai pada indeks top_3011 tanpa mengubah nilai pointer tersebut.

- **Pencegahan Stack Underflow:** Terdapat logika pengecekan jika top_3011 < 0 untuk memastikan program tidak mencoba mengakses indeks negatif saat tumpukan kosong.

4. Metode print()

Metode ini melakukan penelusuran (*traversal*) mulai dari indeks top_3011 hingga 0. Hal ini memastikan tampilan data yang keluar di konsol sesuai dengan perspektif visual sebuah tumpukan, di mana elemen yang terakhir dimasukkan muncul di urutan paling depan/atas.

2.3.4 Class stackArrayDriver_2511533011

```java
package pekan3_2511533011;

public class stackArrayDriver_3011533011 {
    public static void main(String[] args) {
        stackArray_2511533011 s = new stackArray_2511533011();
        s.push_3011(10);
        s.push_3011(20);
        s.push_3011(30);
        System.out.println(s.pop_3011() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah : " + s.peek_3011());
        System.out.print("Elemen pada stack : ");
        s.print();

    }
}
```

Gambar 2.3.4

Class ini merupakan *Main Class* yang berfungsi untuk mensimulasikan penggunaan struktur data Stack yang telah dibuat secara manual.

1. Instansiasi dan Alokasi Objek

Baris stackArray_2511533011 s = new stackArray_2511533011(); menunjukkan pembuatan objek baru di memori. Pada saat ini, constructor pada kelas stackArray dipanggil, yang mengakibatkan variabel top_3011 diatur ke -1 dan array a disiapkan dengan kapasitas 1000.

2. Simulasi Operasi LIFO

Melalui urutan pemanggilan metode, class ini membuktikan prinsip **Last In First Out**:

- **Operasi Push**: Angka 10, 20, dan 30 dimasukkan secara berurutan. Posisi 30 kini menjadi yang teratas.
- **Operasi Pop**: Saat s.pop_3011() dipanggil, angka **30** keluar terlebih dahulu. Hal ini sesuai dengan aturan Stack, meskipun 30 adalah data terakhir yang masuk.
- **Operasi Peek**: Setelah 30 dikeluarkan, peek_3011() menunjukkan bahwa angka **20** naik menjadi elemen teratas yang baru.

3. Visualisasi Akhir

Metode s.print() dipanggil untuk menampilkan isi stack yang tersisa. Output yang diharapkan adalah 20 10. Urutan ini membuktikan bahwa program mencetak data dari posisi paling atas (indeks top) ke posisi paling bawah (indeks 0).

2.3.5 Class StackPostfix_2511533011

```java
package pekan3_2511533011;
import java.util.Scanner;

public class StackPostfix_2511533011 {
    public static int postfixEvaluate_2511533011(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) { // an operand (integer)
                s.push(input.nextInt());
            } else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if(operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);

                }
            }
        }
        input.close();
        return s.pop();
    }
    public static void main(String[] args) {
        System.out.println("hasil postfix = " + postfixEvaluate_2511533011("5 2 5 * + 7 -"));
    }
}
```

Gambar 2.3.5

1. Konsep Ekspresi Postfix

Operator diletakkan **setelah** operand (misal: 5 2 + alih-alih 5 + 2). Stack sangat cocok untuk ini karena kemampuannya menyimpan angka sementara sampai operator ditemukan.

2. Algoritma Evaluasi

Di dalam metode postfixEvaluate_2511533011, terdapat logika krusial:

- **Operand (Angka):** Setiap kali Scanner menemukan angka (input.hasNextInt()), angka tersebut langsung di-*push* ke dalam stack.
- **Operator (+, -, , /):* Ketika ditemukan operator, program akan melakukan dua kali pop():
    - operand2: Angka yang terakhir masuk.
    - operand1: Angka yang masuk sebelumnya.

- **Pentingnya Urutan:** Urutan operand1 dan operand2 sangat penting, terutama untuk pengurangan (-) dan pembagian (/), karena sifatnya yang tidak komutatif ($operand1 / operand2 \neq operand2 / operand1$).

3. Tracing Output 5 2 5 * + 7 -

Langkah-langkah eksekusinya (*tracing*):

1. Push 5, Push 2, Push 5. (Isi stack: [5, 2, 5])
2. Temukan *: Pop 5 dan 2, kalikan ($2 * 5 = 10$), Push 10. (Isi stack: [5, 10])
3. Temukan +: Pop 10 dan 5, jumlahkan ($5 + 10 = 15$), Push 15. (Isi stack: [15])
4. Temukan 7: Push 7. (Isi stack: [15, 7])
5. Temukan -: Pop 7 dan 15, kurangi ($15 - 7 = 8$), Push 8.
6. **Hasil Akhir:** 8.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum pada pekan ke-3 mengenai struktur data Stack, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Prinsip Kerja:** Stack merupakan struktur data linear yang menerapkan prinsip **LIFO (*Last In First Out*)**, di mana elemen yang terakhir dimasukkan akan menjadi yang pertama kali dikeluarkan. Hal ini dibuktikan melalui seluruh pengujian pada 6 *class* yang telah diimplementasikan.

2. **Fleksibilitas Implementasi:** Stack dapat diimplementasikan dengan berbagai cara, baik secara manual menggunakan **Array statis** (seperti pada stackArray), menggunakan **ArrayList** (seperti pada SiswaStack), maupun menggunakan library standar **java.util.Stack**.

3. **Manajemen Memori:** Implementasi menggunakan array memiliki keterbatasan pada ukuran tetap (*fixed size*) yang berisiko menyebabkan *Stack Overflow*. Sebaliknya, penggunaan ArrayList atau java.util.Stack memberikan fleksibilitas karena ukuran memori dapat berkembang secara dinamis sesuai kebutuhan data.

4. **Aplikasi Luas:** Struktur data Stack tidak hanya sekadar penyimpanan data, tetapi sangat krusial dalam menyelesaikan persoalan logika komputasi, seperti **evaluasi ekspresi Postfix** dan algoritma pencarian nilai tertentu tanpa merusak integritas data asli (dengan bantuan stack *backup*).

DAFTAR PUSTAKA

[1] Prasatya, "Apa itu Stack dalam Ilmu Pemrograman," CODEPOLITAN, 04 Juni 2025. [Online]. Available: https://id.scribd.com/document/437845947/Stack-java. [Accessed 18 April 2026].

LAPORAN TUGAS

1. Deskripsi Tugas Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

2. Aturan Penamaan (Wajib Dipatuhi)

1) Nama Kelas : Gunakan akhiran NIM lengkap. Contoh : Website_2311532008 dan Browser_2311532008.

2) Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM. Contoh (NIM ...2008): String judul_2008, String url_2008.

3. Struktur Program

1) Kelas ADT Website (Website_NIMLengkap) Atribut : - Judul Website - URL Website Method : - Constructor untuk menginisialisasi atribut. - - Selektor (Getter) untuk setiap atribut. Mutator (Setter) untuk setiap atribut.

2) Kelas Driver (Browser_NIMLengkap) - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan. - - - Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus. Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan. Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan .isEmpty().

4. Contoh Tampilan Output Program === Browser History NIM: 2411532007 ===
1. Kunjungi Website (Push) 2. Tombol Back (Pop) 3. Lihat Halaman Aktif

(Peek) 4. Keluar Pilihan: 1 Masukkan Judul: SSO UNAND Masukkan URL: https://sso.unand.ac.id/login Berhasil mengunjungi halaman!

1. Class Website_2511533011

```java
package pekan3_2511533011;

public class Website_2511533011 {
    private String judul_3011;
    private String url_3011;

    public Website_2511533011(String judul, String url) {
        this.judul_3011 = judul;
        this.url_3011 = url;
    }

    public String getJudul_3011() {
        return judul_3011;
    }

    public String getUrl_3011() {
        return url_3011;
    }

    public void setJudul_3011(String judul) {
        this.judul_3011 = judul;
    }

    public void setUrl_3011(String url) {
        this.url_3011 = url;
    }

    @Override
    public String toString() {
        return "Judul Website : " + judul_3011 +
               "\nURL Website   : " + url_3011;
    }
}
```

2. Class Browser_2511533011

```java
package pekan3_2511533011;
import java.util.Scanner;
import java.util.Stack;

public class Browser_2511533011 {

    public static void tampilkanMenu_3011() {
        System.out.println("=== Browser History NIM: 2511533011 ===");
        System.out.println("1. Kunjungi Website (Push)");
        System.out.println("2. Tombol Back (Pop)");
        System.out.println("3. Lihat Halaman Aktif (Peek)");
        System.out.println("4. Cek Status History");
        System.out.println("5. Keluar");
    }

    public static void kunjungiWebsite_3011(Stack<Website_2511533011> history, Scanner input) {
        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();

        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        history.push(new Website_2511533011(judul, url));
        System.out.println("Berhasil mengunjungi halaman!");
    }

    public static void tombolBack_3011(Stack<Website_2511533011> history) {
        if (history.isEmpty()) {
            System.out.println("History kosong, tidak bisa kembali.");
        } else {
            Website_2511533011 hapus = history.pop();
            System.out.println("Halaman ditutup:");
            System.out.println(hapus.getJudul_3011());
        }
    }

    public static void lihatHalamanAktif_3011(Stack<Website_2511533011> history) {
        if (history.isEmpty()) {
            System.out.println("Tidak ada halaman aktif.");
        } else {
            System.out.println("Halaman aktif:");
            System.out.println(history.peek());
```

```java
            System.out.println(history.peek());
        }
    }

    public static void cekStatus_3011(Stack<Website_2511533011> history) {
        if (history.isEmpty()) {
            System.out.println("History kosong.");
        } else {
            System.out.println("Jumlah history: " + history.size());
        }
    }

    public static void main(String[] args) {
        Stack<Website_2511533011> history = new Stack<>();
        Scanner input = new Scanner(System.in);
        int pilihan;

        do {
            tampilkanMenu_3011();
            System.out.print("Pilihan: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    kunjungiWebsite_3011(history, input);
                    break;
                case 2:
                    tombolBack_3011(history);
                    break;
                case 3:
                    lihatHalamanAktif_3011(history);
                    break;
                case 4:
                    cekStatus_3011(history);
                    break;
                case 5:
                    System.out.println("Keluar dari program.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
```

```java

            System.out.println();

        } while (pilihan != 5);

        input.close();
    }
}
```

Output :

```
=== Browser History NIM: 2511533011 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511533011 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan:
```

Program ini terdiri dari dua class yaitu Website_2511533011 dan Browser_2511533011 yang digunakan untuk mensimulasikan **browser history** dengan struktur data **Stack (LIFO)**. Class Website_2511533011 berfungsi sebagai ADT untuk menyimpan data website berupa judul dan URL, serta dilengkapi constructor, getter, setter, dan method toString() untuk menampilkan informasi website. Sedangkan class Browser_2511533011 berfungsi sebagai program utama yang menggunakan Stack<Website_2511533011> untuk menyimpan riwayat kunjungan website. Program menyediakan menu untuk mengunjungi website baru (push), kembali ke halaman sebelumnya (pop), melihat halaman aktif (peek), mengecek status history menggunakan isEmpty() atau size(), serta keluar dari program. Program ini menerapkan konsep OOP dan penggunaan stack sesuai prinsip Last In First Out.